

Data Structures

4. Dictionaries

Python dictionaries are an implementation of key-value objects, the keys being indexed.

Keys are often text, values can be of different types and structures.

To create a dictionary, you can proceed by using braces ({}). As encountered in sets, if we evaluate the following code, we get a dictionary :

```
empty_dict = {}  
print(type(empty_dict))
```

Output: <class 'dict'>

To create a dictionary with entries, the braces can be used. Each entry is separated by commas, and the key is distinguished from the associated value by two points (:):

```
my_dict = { "last_name": "Mohammedi",  
            "first_name": "Yahia",  
            "birth_year": 1992,  
            "teams": ["MCA", "JSK"]}  
print(my_dict)
```

Output: { 'last_name': 'Mohammedi', 'first_name': 'Yahia', 'birth_year': 1992, 'teams': ['MCA', 'JSK'] }

4.1. Accessing the values (Extraction of the Elements)

To access the values associated with a key, we use the square brackets ([]):

```
print(my_dict["first_name"])
```

Output: Mohammedi

If the extraction is done by a key not present in the dictionary, an error will be returned:

```
print(my_dict["age"])
```

Output:

```
## Error in py_call_impl(callable, dots$args, dots$keywords):  
KeyError: 'age'  
##  
## Detailed traceback:  
##   File "<string>", line 1, in <module>
```

You can test the presence of a key with the operator in:

```
print(last_name in my_dict)
```

Output: True

```
print(age in my_dict)
```

Output: `False`

The extraction of values can also be done using the `get()` method, which returns a `None` value if the key is not present:

```
print(my_dict.get("age"))
```

Output: `None`

4.2. Keys and values

Using the `key()` method, the keys of the dictionary can be accessed:

```
the_keys = my_dict.keys()
print(the_keys)
```

Output: `dict_keys(['last_name', 'first_name', 'birth_year', 'teams'])`

```
print(type(the_keys))
```

Output: `<class 'dict_keys'>`

It is then possible to transform this key enumeration into a list:

```
the_keys_list = list(the_keys())
print(the_keys_list)
```

Output: `['last_name', 'first_name', 'birth_year', 'teams']`

The `values()` method provides the dictionary values:

```
the_values = my_dict.values()
print(the_values)
```

Output: `dict_values(['Mohammedi', 'Yahia', 1992, ['MCA', 'JSK']])`

```
print(type(the_values))
```

Output: `<class 'dict_values'>`

It is then possible to transform this value enumeration into a list:

```
the_values_list = list(the_values())
print(the_values_list)
```

Output: `['Mohammedi', 'Yahia', 1992, ['MCA', 'JSK']]`

The `items()` method provides keys and values in the form of tuples:

```
the_items = my_dict.items()
print(the_items)
```

Output: `dict_items([('last_name', 'Mohammedi'), ('first_name', 'Yahia'), ('birth_year', 1992), ('teams', ['MCA', 'JSK'])])`

```
print(type(the_items))
```

Output: `<class 'dict_items'>`

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

4.3. Modifying the values

To modify the value associated with a key, we use the square brackets ([]):

```
my_dict["age"] = 29
print(my_dict)
```

4.3. Modifying the values

The values() method provides the dictionary values:

```
the_values = my_dict.values()
print(the_values)
```

Output: dict_values(['Mohammedi', 'Yahia', 1992, ['MCA', 'JSK']])

4.3. Modifying the values

To modify the value associated with a key, we use the square brackets ([]):

The values() method provides the dictionary values:

```
the_values = my_dict.values()
print(the_values)
```

Output: dict_values(['Mohammedi', 'Yahia', 1992, ['MCA', 'JSK']])

4.3. Modifying the values

To modify the value associated with a key, we use the square brackets ([]):

4.2. Adding an entry

To add an entry, we use the square brackets to specify the key, and assign the value to it:

```
my_dict["age"] = 29
print(my_dict)
```

Output: { 'last_name': 'Mohammedi', 'first_name': 'Yahia', 'birth_year': 1992, 'teams': ['MCA', 'JSK'], 'age': 29 }